

IPST-CNAM Albi  
Intelligence Artificielle  
**Master 1 TRIED**

---

**TP noté : Implémenter des jeux  
en situation d'adversité**  
Tic-Tac-Toe : MinMax et élagage Alpha-Bêta

---

Auteur : **Mohamed Amine Sobhi**

Professeur : **M. Adrien Escourrou**

Année universitaire : 2025 – 2026

Juillet 2026

# Table des matières

|          |                                                           |           |
|----------|-----------------------------------------------------------|-----------|
| <b>1</b> | <b>Contexte et besoin</b>                                 | <b>2</b>  |
| <b>2</b> | <b>Le jeu et ses fonctionnalités</b>                      | <b>2</b>  |
| <b>3</b> | <b>Architecture et choix techniques</b>                   | <b>4</b>  |
| <b>4</b> | <b>L'algorithme MinMax avec élagage Alpha-Bêta</b>        | <b>4</b>  |
| 4.1      | Représentation et évaluation . . . . .                    | 4         |
| 4.2      | Principe MAX / MIN et implémentation . . . . .            | 4         |
| 4.3      | L'élagage Alpha-Bêta . . . . .                            | 6         |
| 4.4      | Choix du coup et exactitude des scores affichés . . . . . | 6         |
| <b>5</b> | <b>Visualisation du raisonnement</b>                      | <b>8</b>  |
| <b>6</b> | <b>Tests unitaires</b>                                    | <b>9</b>  |
| <b>7</b> | <b>Installation et exécution</b>                          | <b>10</b> |
| <b>8</b> | <b>Limites et pistes d'amélioration</b>                   | <b>10</b> |
| <b>9</b> | <b>Usage de l'IA</b>                                      | <b>10</b> |
|          | <b>Conclusion</b>                                         | <b>11</b> |

# 1 Contexte et besoin

Dans un jeu à deux joueurs à somme nulle, un programme doit choisir le meilleur coup *en supposant que son adversaire joue lui aussi de façon optimale*. L'algorithme **MinMax** répond à ce besoin : il explore l'arbre de toutes les parties possibles et remonte, à chaque niveau, la valeur la plus favorable au joueur qui décide. L'**élagage Alpha-Bêta** l'optimise en coupant les branches dont on peut prouver qu'elles n'influenceront pas la décision finale.

L'objectif de ce TP est double : implémenter correctement MinMax et son élagage, et surtout **rendre l'algorithme visible** : l'interface affiche à chaque tour le score de chaque coup, le nombre d'états explorés et les branches élaguées.

## ▷ Point clé

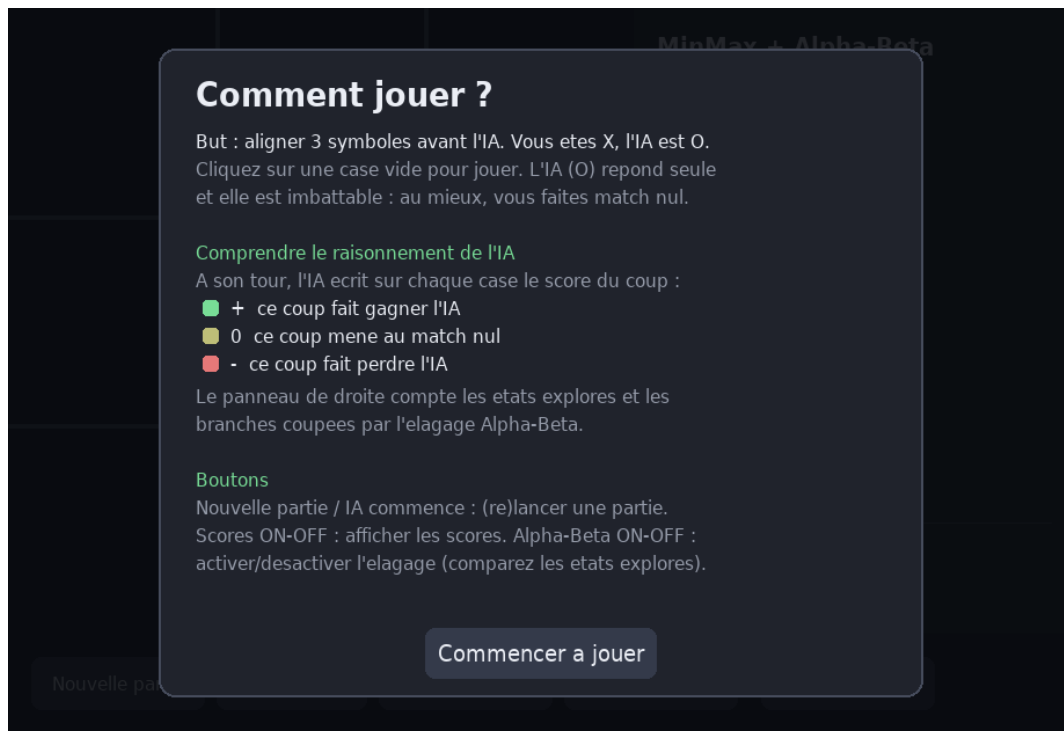
Le morpion 3×3 a été choisi car son arbre est **entièrement calculable** (549 945 états depuis le plateau vide). L'IA joue donc **parfaitement**, ce qui permet de vérifier objectivement l'implémentation : une IA optimale ne doit *jamais* perdre.

# 2 Le jeu et ses fonctionnalités

Morpion 3×3 en fenêtre **Pygame** : le joueur humain (**X**) affronte une IA autonome (**O**) pilotée par MinMax + Alpha-Bêta. L'IA est **imbattable**, le meilleur résultat possible pour le joueur étant le match nul.

- **Score MinMax affiché sur chaque case libre** au tour de l'IA : vert = l'IA gagne, jaune = nul, rouge = l'IA perd ; la case choisie est mise en évidence.
- **Panneau latéral** : nœuds explorés, branches élaguées, profondeur et explication de la décision.
- **Alpha-Bêta ON/OFF** et **Scores ON/OFF** pour comparer en direct.
- Menu « Comment jouer ? » au lancement, temporisation de 4 s avant le coup de l'IA, et **14 tests unitaires** (pytest).

| Action                             | Commande                             |
|------------------------------------|--------------------------------------|
| Jouer un coup (vous êtes X)        | Clic gauche sur une case vide        |
| Aide / règles                      | Bouton <b>Comment jouer ?</b>        |
| Recommencer / laisser l'IA débiter | <b>Nouvelle partie / IA commence</b> |
| Afficher ou masquer les scores     | Bouton <b>Scores ON/OFF</b>          |
| Activer ou désactiver l'élagage    | Bouton <b>Alpha-Bêta ON/OFF</b>      |



**FIGURE 1** – Menu « Comment jouer ? » : règles, légende des couleurs et rôle de chaque bouton

### 3 Architecture et choix techniques

**Python 3** (recommandé par le sujet), **Pygame** pour le rendu 2D et **pytest** pour les tests. Le projet repose sur une séparation stricte entre la logique et l'affichage :

| Fichier      | Rôle                                                             |
|--------------|------------------------------------------------------------------|
| algo.py      | Logique du jeu + MinMax/Alpha-Bêta. <b>Aucun code graphique.</b> |
| main.py      | Interface Pygame + visualisation du raisonnement.                |
| test_algo.py | 14 tests unitaires (pytest).                                     |
| README.md    | Documentation d'exécution + section Usage IA.                    |

#### ▷ Point clé

Comme algo.py n'importe jamais Pygame, les tests s'exécutent **sans ouvrir de fenêtre graphique** : ils sont rapides et automatisables.

## 4 L'algorithme MinMax avec élagage Alpha-Bêta

### 4.1 Représentation et évaluation

Le plateau est une liste de 9 cases ('X', 'O' ou ' '). Un état est **terminal** si un joueur a aligné trois symboles ou si le plateau est plein : c'est la condition d'arrêt de la récursion.

Le score est calculé **du point de vue de l'IA** et **pondéré par la profondeur** :  $+10 - d$  si l'IA gagne,  $-10 + d$  si elle perd, 0 pour un nul.

```
def evaluate(board, ai_player, depth):
    win = winner(board)
    if win == ai_player:
        return 10 - depth
    if win == other_player(ai_player):
        return depth - 10
    return 0
```

#### ▷ Point clé

Sans la pondération, toutes les victoires vaudraient 10 et l'IA n'aurait aucune raison de gagner vite. Ici une victoire en 1 coup vaut 9 contre 5 pour la même victoire en 5 coups : l'IA **gagne le plus vite possible** et **retarde** une défaite inévitable.

### 4.2 Principe MAX / MIN et implémentation

Quand c'est le tour de l'IA, le nœud est un **MAX** (il garde la plus grande valeur); quand c'est le tour de l'adversaire, c'est un **MIN** (la plus petite). Les niveaux alternent en descendant dans l'arbre.

```
def minimax(board, current_player, ai_player, depth=0,
            alpha=float("-inf"), beta=float("inf"),
            stats=None, use_pruning=True):

    if is_terminal(board):                # cas de base
        return evaluate(board, ai_player, depth)

    if current_player == ai_player:      # noeud MAX
        best = float("-inf")
        for move in available_moves(board):
            board[move] = current_player  # on joue le coup
            score = minimax(board, other_player(current_player),
                            ai_player, depth + 1, alpha, beta,
                            stats, use_pruning)
            board[move] = EMPTY           # backtracking
            best = max(best, score)
            alpha = max(alpha, best)
            if use_pruning and beta <= alpha: # coupure Beta
                break
        return int(best)
    else:                                # noeud MIN
        best = float("inf")
        for move in available_moves(board):
            board[move] = current_player
            score = minimax(board, other_player(current_player),
                            ai_player, depth + 1, alpha, beta,
                            stats, use_pruning)
            board[move] = EMPTY
            best = min(best, score)
            beta = min(beta, best)
            if use_pruning and beta <= alpha: # coupure Alpha
                break
        return int(best)
```

Trois mécanismes sont essentiels :

1. **Le backtracking** : on joue le coup, on explore, puis on **remet la case à vide**. Cela évite de copier le plateau à chaque nœud, mais impose de défaire systématiquement chaque coup essayé.
2. **L'alternance** : chaque appel récursif passe la main à `other_player(...)` et incrémente la profondeur, ce qui fait alterner automatiquement MAX et MIN.
3. **La symétrie** : les deux blocs sont le miroir l'un de l'autre. MAX utilise `max` et `alpha`, MIN utilise `min` et `beta`.

### 4.3 L'élagage Alpha-Bêta

Deux bornes descendent dans l'arbre : **alpha**, le meilleur score déjà garanti à MAX, et **beta**, celui déjà garanti à MIN. La règle est unique : dès que  $\beta \leq \alpha$ , on arrête d'explorer les enfants restants.

**Intuition.** Si l'IA a déjà garanti 3 ailleurs ( $\alpha = 3$ ) et qu'un nœud MIN révèle une première feuille à 2, ce nœud vaudra au mieux 2 puisqu'il prend le minimum. Il ne sera donc jamais choisi : inutile d'évaluer ses autres feuilles.

### 4.4 Choix du coup et exactitude des scores affichés

`minimax()` retourne un score, pas un coup. Le choix se fait par une boucle à la racine, qui mémorise en plus **le score de chaque coup**. C'est ce dictionnaire qui est affiché sur la grille.

```
def best_move(board, ai_player, use_pruning=True):
    stats, move_scores = SearchStats(), {}
    best_score, chosen = float("-inf"), None

    for move in available_moves(board):
        board[move] = ai_player
        # Fenetre NEUVE pour chaque coup racine (voir encadre ci-dessous)
        score = minimax(board, other_player(ai_player), ai_player,
                        depth=1, alpha=float("-inf"), beta=float("inf"),
                        stats=stats, use_pruning=use_pruning)
        board[move] = EMPTY
        move_scores[move] = score          # <- affiche sur la grille
        if score > best_score:
            best_score, chosen = score, move

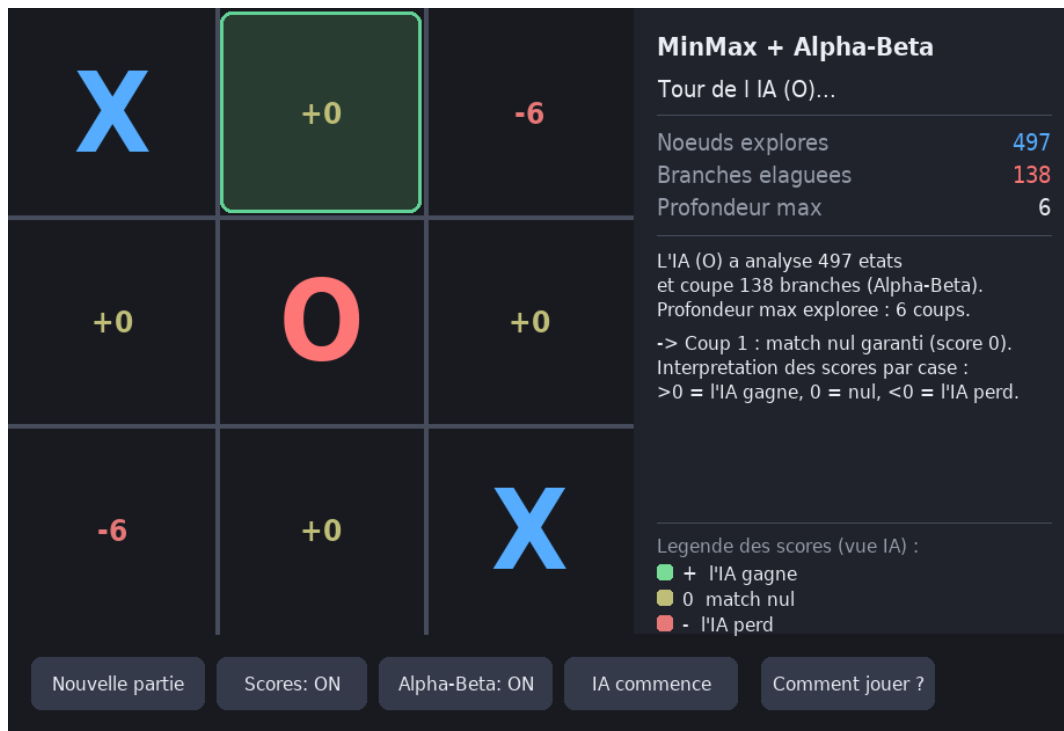
    return Decision(move=chosen, move_scores=move_scores, stats=stats)
```

Un point subtil a dicté ce code. Dans la version classique, on propage alpha d'un coup racine au suivant : c'est plus rapide, mais l'algorithme ne renvoie alors qu'une **borne** pour les coups non retenus, et non leur valeur exacte. Un coup réellement perdant pouvait ainsi s'afficher « 0 » au lieu de « -6 ». Correct pour *jouer*, mais trompeur pour *afficher*. La solution retenue évalue chaque coup racine avec une **fenêtre neuve**, l'élagage restant actif à l'intérieur de chaque sous-arbre.

**▷ Point clé**

Ce choix coûte 34 202 états explorés au lieu de 20 866 depuis le plateau vide, mais reste **16 fois moins** que MinMax pur (549 945), et les scores affichés redeviennent **rigoureusement identiques** à ceux de MinMax pur. Pour un projet dont le but est de *montrer* l'algorithme, l'exactitude prime sur les derniers pourcents de performance.

## 5 Visualisation du raisonnement



**FIGURE 2** – L'IA (O) réfléchit après deux coins joués par X. Chaque case libre affiche son score Min-Max; élagage *activé*

La plupart des coups mènent au nul (0), mais **deux cases affichent –6 en rouge** : les deux coins libres. En jouant l'un d'eux, l'IA se ferait piéger par une **fourchette** : X créerait deux menaces simultanées, impossibles à bloquer toutes les deux. L'IA les écarte et choisit un côté à 0, garantissant le nul. Le panneau indique **497 états explorés** et **138 branches coupées**.



**FIGURE 3** – Même position, élagage *désactivé* : 1 052 états au lieu de 497, pour des scores et un coup choisi *identiques*

| Position              | MinMax pur | Alpha-Bêta | Réduction |
|-----------------------|------------|------------|-----------|
| Plateau vide          | 549 945    | 34 202     | ≈ 16×     |
| Après 1 coup adverse  | 55 504     | 8 465      | ≈ 7×      |
| Position de la figure | 1 052      | 497        | ≈ 2×      |

#### ▷ Point clé

Double message de l'élagage : les **scores et le coup choisi restent identiques**, mais le nombre d'états explorés est fortement réduit. Sur un jeu à facteur de branchement élevé comme les échecs, c'est cette optimisation qui rend la recherche possible en pratique.

## 6 Tests unitaires

14 tests pytest portés exclusivement sur `algo.py`, lancés par `pytest -v` et tous au vert. Ils vérifient notamment que :

- les victoires (ligne, colonne, diagonale) et le match nul sont détectés ;
- l'IA saisit une victoire immédiate, bloque celle de l'adversaire et **préfère la victoire la plus rapide** ;
- l'élagage donne **la même valeur** que MinMax pur, tout en explorant **strictement moins** de nœuds ;
- les **scores affichés sont exacts** même avec l'élagage actif (`test_displayed_scores_are_exact_with`).

— deux IA optimales qui s'affrontent font **toujours match nul**.

▷ **Point clé**

Ce dernier test est le plus fort du projet : au morpion, le résultat théorique d'un jeu parfait des deux côtés est connu (nul), donc tout autre résultat révélerait un bug dans l'algorithme.

## 7 Installation et exécution

```
python3 -m venv .venv && source .venv/bin/activate # Windows : .venv\Scripts\
activate
pip install -r requirements.txt

python main.py      # lancer le jeu
pytest -v           # lancer les tests
```

Prérequis : Python 3.9+. La fenêtre s'ouvre sur le menu d'aide ; après l'avoir fermé, il suffit de cliquer sur une case pour jouer.

## 8 Limites et pistes d'amélioration

- **Taille de l'arbre** : l'exploration complète n'est possible que parce que le morpion est petit. Pour un jeu plus grand, il faudrait **limiter la profondeur** et ajouter une **fonction d'évaluation heuristique** des positions non terminales.
- **Pas de mémorisation** : une position atteinte par des ordres de coups différents est recalculée. Une **table de transposition** y remédierait.
- **Ordonnancement des coups** : explorer d'abord les coups prometteurs rendrait l'élagage nettement plus agressif.
- Autres pistes : animation pas-à-pas de l'arbre, difficulté réglable via une profondeur limitée.

## 9 Usage de l'IA

Conformément au paragraphe 3 du sujet. **IA utilisée** : Claude (Anthropic).

**Pourquoi** : uniquement pour des éléments **périphériques à l'algorithme**, soit une fonction utilitaire d'affichage (retour à la ligne du texte dans le panneau latéral), deux tests de routine sur la détection des alignements, et un appui à la rédaction de la documentation. Le cœur du projet (logique du jeu et algorithme MinMax + Alpha-Bêta dans `algo.py`) a été écrit et compris par mes soins.

**Exemples de demandes :** « Comment gérer le retour à la ligne d'un texte dans un panneau Pygame de largeur fixe ? » ; « Génère des tests pytest couvrant la détection d'une victoire en colonne, en diagonale, et le match nul. »

**Ce qui a été compris et validé :** le principe MAX/MIN, le rôle des bornes alpha/beta, la pondération par la profondeur, et l'équivalence entre MinMax pur et Alpha-Bêta. Validation par l'exécution des 14 tests et par des parties manuelles confirmant que l'IA est imbattable.

**Encadrement :** les blocs concernés (fonction `_wrap` de `main.py`, deux tests de `test_algo.py`) sont délimités dans les sources par :

```
# ##### CODE IA (Claude - Anthropic) #####  
...  
# #####
```

## Conclusion

Ce TP m'a fait comprendre la **nature récursive du raisonnement adversarial** : pour évaluer un coup, il faut supposer la meilleure réponse de l'adversaire, ce qui suppose d'évaluer nos propres réponses, et ainsi de suite jusqu'aux fins de partie.

Le choix de **visualiser** l'algorithme plutôt que de simplement le faire tourner s'est révélé le plus formateur : on voit l'IA écarter les cases menant à une fourchette, et en basculant l'élagage on constate qu'un facteur 16 sur le nombre d'états explorés aboutit à **exactement la même décision**. Cette équivalence entre optimisation et correction est, à mon sens, l'apport théorique le plus marquant du TP.

La limite identifiée, l'impossibilité d'explorer l'arbre complet dès que le jeu grandit, ouvre sur la suite : heuristiques d'évaluation, profondeur limitée et tables de transposition, briques des moteurs de jeu modernes.